# 電　　脳

2026

# 目次


はじめに

78th kazuppa 3

手引き

78th hamu_chan 4

情報科学の達人プロジェクトを受けてみて

78th Unbakedbread 6

より良いコードを書こう

78th Tamiji 8

パソコン甲子園参加記

76th tfl_tkpc 12

読みやすいコードを書こう

78th hamu_chan 18

締め切りを守るには

78th ryupepe 22

パ研新入生コンテスト開催記

78th kazuppa 26

APC の入塾テストと JOI 春合宿の参加記

78th goldenpotato 31

文化祭を支える？ 技術

76th kinugoshi8928 33

おわりに

76th Magentor 39

# はじめに

78th kazuppa

このたびは『電脳 2026』を手に取っていただき、ありがとうございます。パーソナルコンピューター研究部（通称パ研）中学部長の大槻和葉です。

今年も中学部長として「はじめに」を書くことになりました。折角なのでと去年の「はじめに」を読み直したところ、「では、私たちにできることは何か――それは、自分たちもまた次の世代へとこの活動を引き継ぎ、繋げていくことだと考えています」という一文が目に留まりました。

あれから一年、パ研には多くの 80 期が入り、放課後の部室はいつも賑やかになっています。新入生歓迎会の時は「もし 80 期があまり入ってこなかったらどうしよう」と不安になっていましたが、無事に成功しました。

中学生活を通してパ研の活動を先輩から（だいたい）完璧に引き継いだ今、今度は後輩に引き継ぐ番になっていると自分は思っています。

そして、この『電脳』もその一つです。毎年文化祭の部誌として書かれているこれは先輩方の活動の軌跡が記されています。私達先輩は『電脳』に自分の書きたいことを書き、後輩がそれを読む。この流れによってパ研の活動はいつまでも引き継がれていくのではないでしょうか。

今年も（きっと）たくさんの部員が記事を書いてくれていることでしょう。ぜひ、この部誌を読んでパ研の活動の軌跡を皆様が見届けてくださると一部員として嬉しいです。

# 手引き

78th hamu_chan

今年度デコ責の一人である hamu_chan です。部誌の本題に入る前に、繰り返し出てくるワードや、競プロとは何かなどについて書いておいた方が良いと思ったので、書きます。

## 競プロとは？

競プロとは、競技プログラミングの略です。数学に似たような問題が与えられ、それに正当するコードを書き、その正確さやアルゴリズムの質を競います。

「アルゴリズムの質」とはなんでしょうか。例えば、有名な話としては次のようなものがあります。

当時 7 歳の天才数学者ガウスが次の問題を出されたときに、一瞬で答えを求めたというものです。

1+2+3+...+100 は？

地道に計算すると極めて時間がかかりますが、1+2+3+...+100 =

((1+100)+(2+99)+(3+98)+...+(100+1))=101 × 100/2=5050 と変形すれば、一瞬で答えが求められるようになります。

この解き方は、地道に計算していくのに比べて「質の高い」アルゴリズムです。

競プロではさまざまなアルゴリズムを使ったり考えたりし、より高速に動作する「質の高い」アルゴリズムで書かれたコードのみが、問題に正答できるということになるわけです。

## AtCoder とは？

AtCoder とは、国内最大級の競プロのプラットフォームで、世界最大競プロプラットフォームにも数えられることが多いです。案内が部誌のとなりにおいてあるので、興味がある方はぜひそちらも見てみてください。ちなみに社長は筑駒の OB です。

# 部誌の手引き

　この部誌では、パソコン研究部に所属する様々な部員が、まちまちの内容について叙述しています。競プロをやっている人にしか伝わらないような話題もあると思いますが、競プロをやっていなくても読めるものもあるため、いろいろな方に楽しんでいただけるとうれしいです。

# 情報科学の達人プロジェクトを受けてみて

78th Unbakedbread

78 期中 3 の Unbakedbread です。私は情報科学の達人プロジェクトに 4 月から 7 期生として参加しています。ここにはこのプロジェクトについて思ったことを書きます。

## 情報科学の達人プロジェクトとは

情報科学の達人プロジェクトは、国立情報学研究所 (NII) がやっているプロジェクトです。「頭がいい中高生を集めて情報学に触れて、将来に活躍してもらいたい」という感じです。詳細な説明を知りたい人は、調べたら出てくると思うので調べてみてください。

## 半年でやったこと

最初の半年は、各分野の結構すごい日本の教授たちの講義動画 (約 60 分) をたくさん (約 40 本) 見て、幅広い情報科学の知識をつけるフェーズになっています。分野については、競技プログラミングをやっていると自然に身につくアルゴリズム・グラフ理論のようなものもあれば、普段触れることがない機械学習・ロボットのような分野もありました。ただ、どの分野の講義も前提知識が無くても理解できるものだったため、知らないからすっかり分からないということはなかったです。この半年間が終わると、本格的な研究をすることになります。あんまりよく分かってないですが

## 情達の良い所、悪い所

良い所は結構あります。まず、無料で受けることができます。次に、大学とかに行かないと聞けないであろう講義を家で聞けます。メンター制度もあり、大体受講生 5 人に大学の先生が 2 人ぐらいついて、研究に関する相談に乗ってくれたりします。また、研究室訪問というものもあり、東大と京大に行って、研究室の人から説明を受けたり、中を見た

りすることができます。交通費が支給されるので、コレも実質無料です。他にも、情達の他の中高生がしている研究も見ることができます。あんまり悪い所はないですが、強いて言えば講義動画を全部見ないといけないことです。これらの講義動画を全て見ないと次のフェーズに行けないようになっています。

## 選抜を通るコツ (?)

このように参加して損がない情達ですが、選抜があり、毎年 40 人ほどしか通りません。普通に申し込むことはできますが、何か実績があったりしないと落とされてしまう可能性があるようです。ただし、JOI のセミファイナルステージ (旧本選) に行った人には特別なルートが用意されています。具体的には、情達に申し込んだセミファイナル出場者には、日本情報オリンピック委員会が推薦してくれるため、選抜を抜けることができます。私もそれで入りました。多分この推薦は多分来年もやると思います。

## まとめ

JOI のセミファイナルに行けるぐらい強い中高生競技プログラマーは、情報科学の達人プロジェクトに参加することができます (多分)。参加したいけど競プロの実力が足りない！ という人は、ぜひ精進を頑張ってください。この記事を読んで興味を持ってくれた人がいたら嬉しいです。

# より良いコードを書こう

78th Tamiji

## はじめに

突然ですが質問です．みなさんは今，幸せですか？

はいと答える人も，いいえと答える人も，いると思います．しかし，おそらくすべての人は何らかの悩みを持っているはずです．もしかしたらその悩みの中に，以下のようなものが含まれているかもしれません：

- 競プロで解法がわかっても，コードを書くのが遅いせいで高順位が取れない...
- コードが長すぎて，書くのが非常に面倒...

そんなあなたに，今から書く方法を実践すれば，コードの長さがぐっと短くなります！それでは見ていきましょう！

## 注意事項

- 以下のメソッドは競プロなど個人でプログラミングをする場合にしか使えません．共同開発をする場合は各団体のルールに従ってください．
- また，C++ の利用を想定して書かれています．同じアイデアを Python など別のプログラミング言語に適用することは可能ですが，言語構造が異なる場合があるので注意してください．

## 1. スペース・改行は最小限に！

多くのプログラミング言語では，トークンの間に改行やスペースを入れることができます．しかしこれは本当に必要なのでしょうか？

結論から言うと，**入れなくても動くスペースを入れるのは全くの無駄**です．本来ス

ペースを入れる目的は，トークンの区切れ目をわかりやすくするためでしょう．しかし，VSCode という文明の利器の誕生により，トークンの種類ごとに自動的に色分けをしてくれるようになったのです！ それに，スペースを入れるとコードが長くなり，コード全体が右に寄ってしまいます．これでは本末転倒ですね．

ただ，改行を入れないと 1 行の長さが非常に長くなってしまうので，こまめに入れる必要があります．しかし，なるべく入れたくないので，以下のようにすることをおすすめします．

- ひとつの処理の間は改行しない．
- 短い処理が続くとき (例えば `ll a;cin>>a;` など) は 1 行にまとめる．ただし，エディターの 1/3 くらいを超えたら改行する．
- 閉じる中括弧の後は改行しない．

こうすれば，目を動かす距離を長くせずに改行を最小限にすることができます！

## 2. 変数名は全て 2 文字以下！

あなたは競プロで普段，どのような変数を使っていますか？ `ans`,`sum`,`tot` とか…

**変数名が長すぎます！**

というわけで，変数名は全て 1 文字… は非現実的なので，2 文字以下にしましょう．といっても，a から順番にアルファベットを使うなどでは，流石に混乱してしまいます．そこで以下の方法を使いましょう！

### 2-1. 大文字小文字の使い分け

1 文字といっても大文字と小文字があるわけです．そこで，**数値や `char` を小文字，コンテナを大文字**にするのはどうでしょうか？ これはかなりわかりやすいと思います．もちろん逆でもよいですが，数学の文脈では集合などを大文字で表記することが多いので，個人的にはこちらの方がわかりやすいかと思います．

### 2-2. 用途ごとに使い分けよう

何も考えずに 1 文字変数を使えば，すぐに被ってしまいます．それに VSCode は容易には被りのチェックをしてくれません．しかし，使い分けさえすれば被りのリスクが激減します！

私がおすすめするのは，以下のような使い分けです．

- 入力の長さなどは `n,m,k`
- 区間の左端・右端は `l,r`
- グラフは `G,H`
- for ループのインデックスは `i,j,x,y,z`
- `(a,b,c,d),(p,q),(s,t),(u,v,w),(x,y,z)` などはまとめて考える，つまりどれか 1 つを使ったら同じグループのものを優先的に使う
- たとえば `a` が使えなくなったとき，`a0,a1,a2,...` を使ったり，アルファベットを結合したりする．後者の場合，最大値を表す `mx` のように意味がある程度わかるようにしておくとよい．

こうすれば，一般的な変数の表記法をある程度保ったまま，多くの変数を 1 文字にすることができ，また被りのリスクも最小限に抑えることができるのです！

### 2-3. 多く使う変数を重視

変数には当然，多く使うものと少なく使うものがあるわけです．当然，**前者の変数名を長くし，後者の変数名を短くする**ことが最適です．

たとえば，入力で点 $(A_i, B_i)$ が与えられるが，実際には座標変換などの処理により，点 $(A_i + B_i, A_i - B_i)$ として考えた方がよい場合を考えましょう．このとき，入力を受け取る際は `ll a0,b0;cin>>a0>>b0;` として，そこから `ll a=a0+b0,b=a0-b0;` という処理をするのがよいでしょう．

## 3. テンプレートを整備せよ！

`std::vector` や `for` ループなど，よく使うのに名前が長い機能が，プログラミング言語にはたくさんあります．しかしそれらは変数名ではないので 2. のような工夫は使えま

せん．どうすればいいのでしょうか？

答えは簡単，**テンプレートで型短縮やマクロを事前に定義しておけばよい**のです！ 特に以下のテンプレートは競プロにおいて非常に有用だと思います．

- `using ll=long long;`
- `#define rep(i,n)for(ll i=0;i<(n);++i)`
- `rep` マクロの逆順・区間版など
- `#define all(x)x.begin(),x.end()`
- `#define rall(x)x.rbegin(),x.rend()`
- `#define uniq(a){sort(all(a));a.erase(unique(all(a)),a.end());}`
- `#define lwb(A,a)((ll)(lower_bound(all(A),a)-A.begin()))`
- `#define upb(A,a)((ll)(upper_bound(all(A),a)-A.begin()))`
- `#define maxe(A)(*max_element(all(A)))`
- `#define mine(A)(*min_element(all(A)))`
- `#define acc(A)(accumulate(all(A),OLL))`
- `template<class T>using vec=vector<T>;`

ここで，2. の規則を遵守していれば，全ての変数名は 2 文字以下なのでした．しかし `ll` 以外のテンプレートは全て 3 文字以上なので，**絶対に被ることはありません！** `ll` も普段使いしていれば被ることはないでしょう．

# おわりに

以上，競プロでコードを短くするための 3 つのメソッドをお伝えしました．いかがでしたでしょうか？

正直，ここまで短縮されたコードを書きたくないと思っている方もいると思います．しかし，その苦しみを乗り越えた先に，楽しい競プロ生活が待っていると信じています．更なる高みを目指して，これからも競プロを続けていきましょう．

# パソコン甲子園参加記

76th tfl_tkpc

こんにちは。高校パ研副部長の永重 (tfl_tkpc) です。自分は中学の頃から競技プログラミングをゆる〜く続けていて、Web 開発や機械学習など他の分野にも片足を突っ込んだりしています。

今回は 9 月上旬に行われた「パソコン甲子園」予選の参加記をお届けします。ところどころ補助的な説明も入れているので、競プロ経験者以外の方にもコンテストに参加する雰囲気を少しでも感じてもらえると嬉しいです。

## パソコン甲子園について

競技プログラミングのコンテストといえば、数学などと並んで科学オリンピックの一つに数えられる日本情報オリンピック (JOI) 及び、JOI で選出された日本代表が参加する国際情報オリンピック (IOI) が最も大規模で有名な大会です。しかしこの他にも、有志の団体や企業によってさまざまなコンテストが開催されています。

パソコン甲子園は会津大学が毎年主催する高校生・高専生向けの大会で、競技プログラミングの問題を解くプログラミング部門とアプリの企画・制作を行うモバイル部門の 2 つに分かれています。各部門で上位に選ばれたチームは実際に会津大学に赴き、1 泊 2 日で本選に参加します。自分は今年がパソコン甲子園初出場で、プログラミング部門に kinugoshi8928 君と 2 人のチーム Reply で参加しました。Paken からは同級生の Magentor 君 (高校部長) と highligher_math 君のチームも出場しています。

## コンテスト前日まで

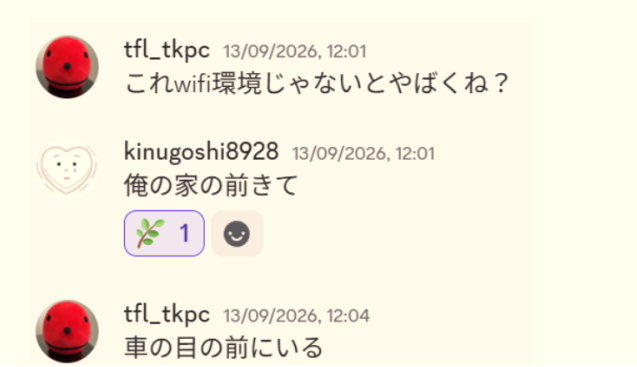
夏休み中はあまり競プロの精進ができていなかったのですが、AtCoder(筑駒 OB が立ち上げた、競技プログラミングのサイトでは規模最大級のサービス) 上でほぼ毎週開催されているコンテストには参加するようにしていました。AtCoder では難易度別に 3 種類 (細かく分けると 5 種類) のコンテストがあり、このうち初級者〜中級者向けの Beginner Contest は毎週、より上級者向けの Regular/Grand Contest もそれぞれ月に 1〜2 回開催

されています。競プロで力をつけるための演習は、AtCoder でコンテストへ参加したり、過去のコンテストの問題を解いたりするのがメインになります。

## コンテスト当日

パソコン甲子園の予選はオンラインの競技システムを用い、各学校で教員などの監督者のもとで行うことになっています。余裕を持って早めに家を出たのは良いのですが、相方の kinugoshi 君に連絡しても一切返信がなく、(いつも通り) 大寝坊したのではないかと心配して彼の家に直接向かいました。幸いなことに (??)、kinugoshi 君は寝坊ではなく、もとから 11 時半に起床してくる"予定"だったそう。

AtCoder などのコンテストサイトでは書いたコードをサイト上で実行できるコードテスト機能がありますが、今回の競技システムは最低限の機能しかついていないので、コードを手元の PC で書いて実行するのに予めセットアップ作業を行う必要があります。これに加えて、コードを書きやすくしたりコンテスト中の手間を減らしたりする様々な機能を導入することをまとめて「環境構築」といいます。普段は家にあるデスクトップ PC からコンテストに参加しているので、ノート PC の方にはまだコードの実行環境を作っておらず、当日まで忘れていました。そこで強硬策として、kinugoshi 君に家の Wi-Fi を貸してもらい、玄関の外で立ったまま PC を開いて必要なシステムをインストールしました。




住宅地のど真ん中だったので近隣住民からものすごく不審な目で見られていそう…

もう一つ忘れていたのはライブラリの準備でした。競プロでは頻繁に出てくるアルゴリズムがたくさんありますが、特に複雑なもの (セグメント木や mod 演算など) は毎回コンテスト中に 0 から実装すると時間がかかって面倒なうえ、実装にミスがあってバグった際に「解法はわかったのに実装が間に合わない」致命傷になりかねません。そこで、予めア

ルゴリズムを実装しておいて、コンテストの際には使用したいアルゴリズムの関数を呼び出して値を代入するだけでよい、という手法が望ましいです。アルゴリズムがたくさん用意してあるものをライブラリと呼び、AtCoder が提供する AtCoder Library(ACL) のほか、よりマイナーなアルゴリズムにはネット上で公開されているものや自作のものを使うこともできます。また、これとは少し違う手法として、自分で頻繁に使うアルゴリズムやキーワードの短縮形での定義などをまとめておいて、コードの冒頭に毎回貼り付ける「テンプレート」も一般的です。

AtCoder などでの一般的なコンテストのほか、JOI の本選 (去年からはセミファイナルステージと呼ばれている) まででは事前に用意したライブラリを参照して使用したり、ネット上で検索したりすることが許可されています。一方、パソコン甲子園ではいずれも禁止となっていて、紙資料の持ち込みのみ許されています。そこで、自作したライブラリを直接印刷したものを持参し、必要に応じてアルゴリズムのコードを丸写しする、という手法が一般的にとられています。アルゴリズム自体の実装はその場で行う必要がありますが、事前にバグが発生しない (はず) だと確かめられるのでその信頼だけでも大きいといえます。今回は kinugoshi 君のライブラリ及び、75 期の keisuke6 先輩が昨年のパソコン甲子園本選で使用し、自身ブログ上で公開してくれていたライブラリを印刷しました。(k6 さんありがとうございます！ ) ちなみにコンテスト直前に気づいて手遅れでしたが、Paken の部室にも置かれている競プロの各種教本も別に使ってよかったんですね。

11 時半起床までは良かったのでしょうが (?)、A4 用紙計 30 枚のライブラリを 2 人分印刷するのに想定以上に手こずる (印刷が途中で止まったらしい) などして、学校に到着したのは予選競技開始のおよそ 12 分前でした。30 分前に集合と監督の先生には言われていたのですが、ただ単にこちらの時間の見通しが甘くて反省。

コンピュータースペースでは既にもう片方のチームが準備を進めていました。ノート PC を学校の Wi-Fi と充電ケーブルに繋ぎ、考察メモ用の筆記用具とライブラリと軽食を手元に用意して、予選競技用の特設サイトを開きログインして、手元でコードを実行できることを確認する... と準備していたら競技開始時刻ギリギリでした。家から参加するオンラインのコンテストにも言えることですが、5 分くらいは余裕を持ちましょう！ 初動が数十秒遅れただけでも、順位が最終提出の時間にも依存するコンテスト (特に大人数が参加する AtCoder) では順位が下がりかねません。

何はともあれ、いざ実戦、頑張るぞい！

# コンテスト中

予選は 3 時間以内に 12 問を解く形式で、2 人チームのそれぞれが個人戦で得た得点の合計で競います。100 点満点ですが、前半の問題は配点が 1〜6 点と非常に小さく、逆に後半は 1 問で 15〜20 点が得られる、という極端な傾斜配点になっています。なんと最終問題が 36 点という恐ろしい回も過去にありましたが、今回もそれほどではないにせよ 12 問中前半 6 問で 2 割程度、9 問目まですべて正解してやっと半分でした。

また、一般的なオンラインコンテストでは不正解のコードを提出すると最終提出時間の値に一定時間のペナルティが加算される (AtCoder は 1 回 5 分) のですが、本コンテストでは同一得点のチーム内では不正解 (ペナルティ) の少なさで先に順位を判断し、最終提出時間はその後の指標になっています。なので、1 点問題だからといって気を抜いて不正解を 1 回出してしまえば致命傷になります。罠として、競技システムでコードを提出する際に毎回コードの言語を Java・Python・C++17 から選ぶのですが、既定値が Java になっているので万一これを変え忘れてしまうとペナルティが付きます。提出前に内容確認画面まで用意されているのでこれに引っかかる人はほとんどいないと信じたいですが...

開始時刻の前後で学校の Wi-Fi 接続がだいぶ不安定だったので、一旦すべての問題の pdf をパソコンにダウンロードしてから問題を解き初めました。序盤の問題は四則演算だけを要求するものから始まり、全探索や簡単なデータ構造の当てはめなど、前半 6 問はある程度競プロを触っていれば容易に解けるレベルの問題でした。とはいえペナルティを踏むのは怖いので、一問ごとに入出力例のサンプルや自明な例を手元のコードで実行して出力があっていることを逐一確かめていました。

続く 7 問目を解こうとしたところ、まさかのここで思いついた解法がコードの実行時間制限に間に合わなさそうで一旦パス。8 問目はわりあいすぐに、9 問目は一度コードの方針を間違えたのち軌道修正してこちらも正解し、この辺りで最初の 1 時間が経過しました。ここからは配点の重い後半 3 問のどれかは通したい思いで、比較的問題がシンプルで取り組みやすそうな 11 問目の考察を進めていましたが、思っていたよりも手が出ず停滞していました。

どんな問題でも考察にあたっては入出力例や小さい例で手を動かして考えてみるのがとても有効です。自分の場合は、数列や文字列に操作を行う問題では編集がしやすいメモ帳、グリッドやグラフなど図示をしたい問題は紙とシャーペンと、なんとなく使い分けていますが、アナログな紙の方がなんでもメモできるので優秀だと思っています。ちなみにメモは印刷してもらったライブラリの紙の裏面に殴り書きをしていました。(直前準備の

項で考察メモ用に筆記用具しか用意していないのは伏線... ということにしたいですが普通に焦っていてルーズリーフ罫線紙を鞄から取り出すのを忘れていただけです。) あと、ここまでの簡単に解ける問題は実装が面倒なアルゴリズムを要しなかった一方、後半の問題は解法の方針も見えなかったので、なんとライブラリはテンプレートとして書く最初の数行を参照する以外一切使用しませんでした。相方も同じだったらしいです。(ライブラリ持参のことを二人とも一切忘れていれば余裕を持って開始に間に合ったのでは???)

2 時間ほど経過したタイミングで 11 問目の解法の糸口が未だ見えず、既に集中力が切れかけていたので一回 7 問目に戻って制約を見直していたところ、実装の仕方を少し工夫するだけで愚直な方針でも正解できることに気づきました。ここまでで前半 9 問を全問正解して 50 点で、用心深さと多少の運のおかげでノーミスです。そのあとは 10 問目から 12 問目を移動しながら考察を進めようとしていましたが、成果は何も得られず 3 時間の予選競技が終了しました。kinugoshi 君からはカフェインを摂取することを強く強く推奨されていましたが、エナジードリンクをわけもなく敬遠するタイプの人間なので炭酸飲料だけ買って前半のうちに飲み切り、結局後半戦は眠かったです。

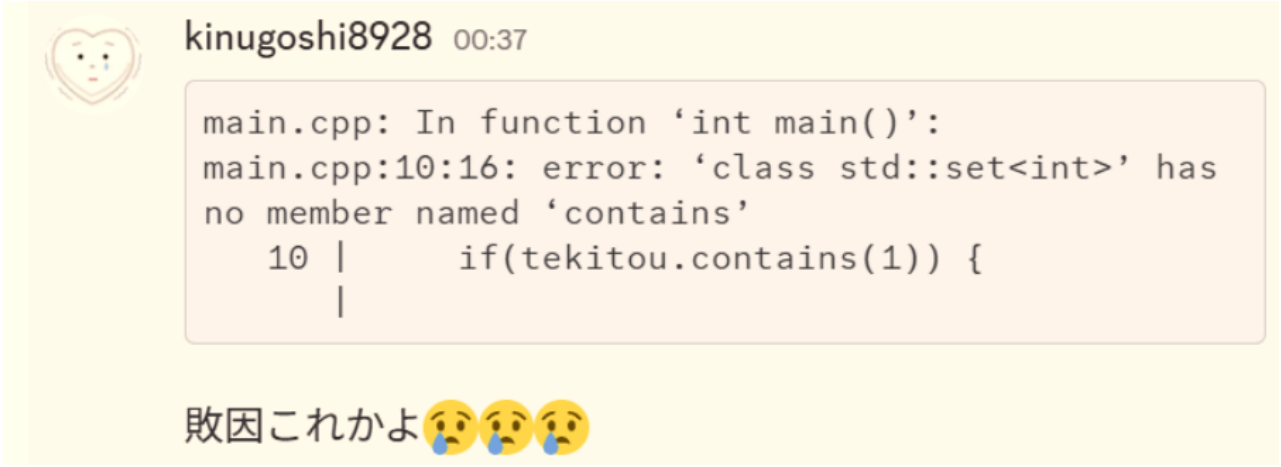
コンテスト終了後

順位表の存在を知らなかったので、kinugoshi 君は 10 問目まで全問正解の 64 点を 2 ミスでとっていることを終わってから初めて知りました。また、一見面倒な 10 問目の方が簡単に解ける一方で、11 問目で使用するべきアルゴリズムはそもそもちゃんと理解していないものだったことがわかり、考察を粘る問題を間違えたことを後悔しました。

会津で行われるパソコン甲子園本選に参加できるのは優秀な成績を収めた 30 チーム程度です。このうち上位 10 チームは順位表で上からそのまま予選通過できるのですが、残り約 20 チームは地域性および女性参加を考慮して選ばれます。そのため、ただでさえ競争が激しい東京都で同じ学校から 2 チームが本選へ参加するのは上位 10 チームに入らないと厳しいと予想していました。

筑駒から出ているもう片方の Magentor-highlighter_math チームはもともと競プロの実力が自分たちより圧倒的に上なこともあり、1〜11 問目と 1〜9+11 問目をそれぞれ正解して 146 点、ペナルティ数はなんと 2 桁出していたものの高得点で本選進出は確実でした。他方、自分たちのチーム Reply は 50+64=114 点で、予選終了 30 分前地点の順位表および SNS 上での報告の情報をもとに考えると上位 10 チームの当落線上にいるだろうということがわかりました。

2 週間後の体育祭 1 日目午前、プログラミング部門の本選進出チームがやっと発表されたのですが、残念ながら自分たちのチームは残っていませんでした。kinugoshi 君は相当落胆していたのですが、本人から以下のコメントを頂いております：




軽く解説すると、kinugoshi 君の手元環境で使っていた最新版の C++23 では’contains’という関数があるのですが、予選競技システムが使っているのは何代か前の C++17 でこの機能は使用することができず、結果として手元では問題なく動くのに実際にはエラーが出て不正解、という勿体ないペナルティを 1 回していたそうです。上位 10 チームの中には同率 114 点のチームが複数あり、kinugoshi 君側の 2 ミスを 1 つでも減らせば本選に行けていた可能性があるだけにかなり悔しい、という顛末です。

## まとめ

せっかくならもう片方のチームの高校部長に本選の参加記も書いてもらいたいところですが、本選が開催されるのはこの筑駒文化祭が終わってから次の週末です。文化祭直前期の過労の反動で風邪を引いたりせず、会津で頑張ってきてほしいです。一方で大本命の日本情報オリンピックもこの先待っているので、自分は文化祭が終わったら競プロの精進を一層頑張りたいと思います。

これは競プロに限った話ではないと思いますが、力をつけるにはコンテストに出まくって、「当たって砕けろ」からでもいいので場数を積みつつ、わからなかった問題は解説を読んで自分で実装することで勉強する、というのがとにかく強力です。競プロについてはコンテストが毎週のように開催されているので、特にこれがしやすい環境にあるのは有難いです。

# 読みやすいコードを書こう

78th hamu_chan

## はじめに

皆さんこんにちは。パソコン研究部中学 3 年で、今年度のデコ責をやっている hamu_chan です。僕からは、「読みやすいコードの書き方」について書きます。部誌に何を書くか迷っていた時に、Tamiji が一文字変数の宣伝をすると聞き、それに対抗して書かれた記事となります（一部内容が真反対ですが、まあ競プロをやっている人の中で意見がわかれるものと考えてもらえると）

さて、読みやすいコードを書くメリットとはなんでしょうか。「美しい」というのもありますが、競プロをするうえでこのメリットといえば、デバッグがしやすくなること。これにつきます。よってこの部誌では「読みやすいコード」を「競プロにおいてデバッグのしやすいコード」と定義して、僕の思うこの特徴について書いてみようと思います。

## 注記

繰り返しになりますが、ここで論じるのは「（僕の思う）競プロにおいてデバッグのしやすいコード」とは何かです。よって以下のことに注意してください。

・そもそもコードの書き方には様々な流儀があり、僕のいう「読みやすい」コードが、全ての人（競プロ er に限定しても）にとって「読みやすい」とは限らないこと

・1 秒を争う競技である競プロにおいて、「読みやすい」コードを書くことが必ずしも最適とは限らないこと

・業務などでの良いコードについては別として考えてほしいということ

また、多くの部分は様々な言語において通用する内容だと思いますが、言語として C++ を想定しているため、一部 C++ 特有の内容が含まれているかもしれません。

## こまめに関数にわける（重要度：★★★）

これは非常に重要だと思っています。まず、同じ処理を二度以上やるときには、毎回書くよりも関数にまとめた方が楽です。デバッグももちろんこの方が楽です。さらにいえば、処理が一度であっても、関数で別途にわけた方がロジックが見やすくなることは多々あります。実装の際のミスも減り、「コードのどの部分で何をやっているか」が明瞭でわかりやすくなります。

## コメントを書く（重要度：★★★）

コメントを書くことによって、この部分のコードが何をしているのかが明瞭になります。短いコードではあまりいらないことも多いですが、100 行程度のコードを書く場合には、コメントをはさんだ方がわかりやすいように思います。

## 半開区間で統一する（重要度：★★★）

一応説明をしておくと、整数列 1, 2, 3, 4, 5 を、[1, 5] と表すのが閉区間、(0, 6) と表すのが開区間、[1, 6) と表すのが半開区間です（正確にはこれは整数のみを要素として考えたときのことです）。競プロにおいては半開区間を用いるのがわかりやすいとされることが多いです。いずれにせよ、同じコードの中で区間の表し方が複数あるのはバグのもとです。絶対に統一しましょう。なお、同様の理由で、添字を for 文で動かしていく場合などでも、そのままなのか、1 足すのか／引くのかなどを、似たことをやっている部分ではある程度統一しておくのが吉です。

## 変数名に規則性を持たせる（重要度：★★☆）

変数の役割と名前をある程度対応づけることで、その変数が何をしているのかが直観的にわかりやすくなります。しかし、規則性のない一文字変数を多用することのデメリットはもう一つあります。変数名衝突のリスクです。コンパイルエラーで検出されればよいの

ですが、スコープの段階が違うなどすると検出されず、予期せぬ挙動を招くことが少なくありません。数文字長くして意味のある変数名にすれば、見やすくなるうえに衝突のリスクも格段に減ります。若干タイプ数が増えますが、所詮数秒の差にしかならないため、メリットの方が大きいと考えます。

## 一行が長くなりすぎないようにする（重要度：★★☆）

単純に、コードを見るときに左右に表示範囲を動かす必要があり、面倒です。C++ ではスペースを入れられる場所には基本的に改行も入れられる（コンパイルエラーとならない）ため、適当な場所で改行するのが良いでしょう。

## 適宜スペースを入れる（重要度：★☆☆）

コードの中に適宜スペースが入っていた方がパッと見たときに読みやすいです。しかし僕はこれに関しては、競プロにおいてはさほど重要ではないように思っています。スペースを入れることによってコードのロジックがわかりやすくなるかというとそうではない（と思う）からです。なお、僕は個人的な美意識の問題でコードにはスペースを入れるようにしています。

## 激長テンプレートを貼らない（重要度：★☆☆）

上位勢のコードにしばしば見られる、長ければ 1000 行にも及ぶコード群のことです。大部分が不要であるコードが前の方に長々と貼られているのは、単純に邪魔ではないでしょうか。しかしながら、多くの上位勢が激長テンプレートを貼っているということは、この主張に賛同してくれる人はわずかなのだと思います。

# おわりに

以上の「読みやすいコードの特徴」たち、いかがだったでしょうか。賛同してもらえる内容もそうでない内容もあったかもしれませんが、デバッグしやすいコードを書くことは競プロにおいて非常に重要です。それでは、最後まで読んでくださりありがとうございました。

# 締め切りを守るには

78th ryupepe

## はじめに

こんばんは (編集時間)、78 期の ryupepe です。提出締め切りが残り 30 分で内容を決めていないので、締め切りを間に合わせる方法について考察します。30 分を寝て過ごすよりはよほど生産性がありますね。

## 方法 1: 気合

そもそも締め切りぎりぎりまで作業を怠っていたのが原因なので、自戒して気合で間に合わせましょう。間に合わせるには以下の方法が考えられます。

### 1. 前回のものを流用する

レポートやプレゼンなどであれば、前回作ったものが残っていることが多いです。コピペして、適宜変更するだけで完成できます。フォーマットから決めるよりよほど時短になります。

### 2. せいせい ai にまかせる

**競プロ er が嫌いがちなせいせい ai に任せるとなんと数秒から数分でレポートが完成します。**とても現代的な案ですね。しかし、この方法には問題点がいくつかあり、学習的効果がない、思考力が欠如するといったものです。何事もほどほどに使いましょう。表を作るときに時短のために使うぐらいがちょうどいいと思います。ぼくはレポートにせいせい ai は使いません。プライドが...

### 3. 友達に頼み込む

友達が書いたものを譲り受けて参考にします。まるまるうつすのはせいせい ai 同様の観点からオヌヌメできません。実験データとかを得るのには手っ取り早いので、レポートそのものでなくともお願いするのが吉です。

### 4. プログラムを組む

たとえば数学であれば、割合を実験での成功回数の割合とみればプログラムでごり押せます。ほかにも計算回数が積めるプログラムには多種多様な使い方があるので、皆さん活用してみましょう。

## 方法 2: 延長

締め切りまでに出すのがどうしても無理そうな場合は、締め切りを伸ばしてもらうことを検討しましょう。評価は下がるでしょうが、出さないよりましです。

### 1. 提出先に頼み込む

例えば授業のレポートならば先生に、部活の調査ならば部長に期限を延ばしてもらえないか頼みましょう。常用するのはよくないですが、数回であれば許してくれる可能性が高いです。

### 2. 無断でだす

提出箱が開いてるから過ぎてても提出しちゃえ!というものです。仮に出せたとしてもよくない方法であることは言うまでもないでしょう。出したあとでもいいので報告する方がリスクは少ないです。しかし、物理的な箱であったり締め切りを甘く見ている人であれば多少は許されるかもしれません。最初のうちはやらない方がいいでしょう。

### 3. 提出サーバーをハッキングする

提出期限が機能しなければ問題ないです。ハッキングして期限を無効化、もしくは延長しましょう。ハッキングで重要なのはサイトのセキュリティの穴を見つけることです。windows であれば F12 から開発者モードでコードが見れるので、穴を探しましょう。Mac ユーザーじゃないので Mac は知りません。

## 3. 中断

期限まで粘って、期限になったら途中でも提出するというものです。途中でふっと終わってしまったら心配になると思われるので、最後に間に合わなかったの一言ぐらい入れておきましょう。一応あがく方法はあります。

### 1. 意味不明な文字列を最後に載せる

見た人は作成中に何かあったのか不安になります。プログラム等でランダムな文字列、文字化けっぽい文字列などを乗っけて相手の心を揺さぶりましょう。もしかしたら後で続きを書かせてくれるかもしれません。

### 2. 洗脳する

提出先の人を洗脳すれば中断したことなど気にされません。文章の最後に相手を洗脳できるような図をおきます。そこで記憶がなくなれば最後まで書いたと思い込みその通り見てくれますよね!だれかやってみてください!!

## 4. あきらめる

提出をあきらめるという方法です。長い間提出するのを忘れていた時や自由課題であるときに有効です。しかし、ダメ元でもいちど交渉してみたほうがいいとは思います。

# おわりに

　ぼくは結局 30 分以内に書き終えられなかったので頼み込んでいます。これが受理されることを信じて...

**みなさんは締め切りに余裕をもって提出しましょう！！ 結局はそれが一番です！！**
ぼくは締め切りぎりぎりにならないとやる気が出ないのでだれかやる気が出る方法を教えてください。

ところで、今回プログラムに多く頼っていますね。みんな競プロやろう!

# パ研新入生コンテスト開催記

78th kazuppa

改めましてこんにちは、kazuppa です。8 月の上旬と下旬、yukicoder にて「Paken 新入生コン」を開催して 2 回合わせて 20 問の新規問題を公開しました。色々なことは X にてポストしましたが、裏話についてはあまり語っていなかったのでここに裏話含め色々書こうと思います。

## コンテストを開催することになった理由

元々 Paken では AtCoder Weekday Contest（AWC）の過去問を使って新入生対象の疑似コンテストを行っていました。しかし、AWC の問題文は全体的に長く、読解も容易でないため問題を解くアルゴリズムを考えて解く以前で詰まってしまうという問題がありました。かといって普段私たちが参加している AtCoder Beginner Contest の過去問を使うと既に解いたことがある人がいる可能性は否めません。そこで、「いっそのこと完全新規で作問をすればいいのではないか」と思いたち、作問を始めることになりました。

## day1

書きたい問題を書きます。また、これ以降は問題のネタバレを含むので十分気を付けてください。

### C,J

間違いなくこのコンテストの問題児です。もともと「簡単な dp 枠」として J の N <= 5000 版を難易度 2.5 で提案していました。そしてそのついでで C が割ときれいな解法が見えたのでこちらも提案することにし、day1 に両方を配置しました。しかし、いざ J の想定解を書いてみるとあまりに実装が大変な量になり、またたくさんバグらせたため難易度 3 に引き上げました（この時に N <= $2 \times 10^5$が解けたので制約をそれに変更しました）。コンテストの前日くらい、マルチテストケースにしてリジャッジを書けたところ僕

の想定解が間違っていることが判明しました。が、どうせすぐ直るだろうと他問題の作業をしていました。コンテスト当日朝、頑張って直しますが一生 $WA$ が取れません。そのまま昼になり牛丼を食べながらスマホでバグを探していたのを覚えています。なんとか $AC$ を得ることができたのがコンテストが始まる 40 分前、そのままダッシュで他問題の作業を行っていました。作業を後回しにすると痛い目を見ることになるので気を付けてください。結果僕の私怨ということでこれを難易度 3.5 に上げることにしました。結果 3.5 でよかったと思います。また、これと同時に $C$ もマルチテストケースにしていたのですが入力を直すのを忘れていました。申し訳ございません...

## E

不変量を取り扱う問題が欲しい！ となりましたが、いくら考えてもそれっぽい問題が思いつかなくて Bread にお願いしたところ、これが返ってきました。難易度を 2 にするか 2.5 にするかは考えていましたが、（見えにくいとはいえ）割と有名な典型だと思ったので 2 にしました。タイトルは僕が JOI-like なやつを提案して採用してもらいました。

## G

このコンテストの少し前に公開した「Min of Max of Div of Sum」にて小数誤差の話に関する説教を大量にいただいたので反省の意思表示として mod 998244353 から小数に変更し、ちゃんとしたジャッジを用意しました。申し訳ございませんでした...

## H

自分はもともと 6N+1 頂点にしてダイクストラを回したせいで「海上で 0 分で無から MT 免許を取得する」というあり得ない状況を起こしていました。すぐ気づいて修正し、それを落とすケースを 1 ケースだけ追加しました。その結果多くの人が 1WA をしていてみんな同じことするんだ... となりました。（知り合いの赤コーダーさんは 8(N+2) 頂点を持っていてええ... となっていました）

### I

元ネタは僕が好きなゲームです（これ以上書くとネタバレになりそうなので書きません）。10^18 回も操作するの大変ですね。この問題、まともな実装量で解く方法が思いつかなくて実装が大量にバグりました。解説も書くのが相当大変でした。まともな解法があれば教えてくださると大変助かります。部分点で誘導していましたが、リアタイしていた人が Mo's Algorithm を取り出していて裏ではい？ になっていたのを覚えています。

## day2

### 全体

同じような元ネタから生えた問題が全体で 5 組もあり、そのうちの 3 組が day2 に移動しました。そのため「単発、同じ元ネタ、単発、同じ元ネタ、単発、同じ元ネタ、単発」とすればいい感じのバランスになると思い、また特別な配点もやってほしいと思ったので全問 100 点 + 難易度非昇順という JOI レベル 100 みたいなコンテストになりました。（実際のところ A,B,C がいい感じになったので同じ元ネタ、単発、単発から始まっています）

### B

ハッシュ使うだけとして今の制約で提案していましたが、3 日前くらいに愚直が通ると言われました。そのため制約を N,Q <= $2 \times 10^5, |t| <= 10$ に変更していました。しかしあり得ないくらいに実行時間がかかっていました。そのために制約を少し小さくしましたが、また別の解法がまた通ってしまうという事態に陥ってしまいました。そのため、最終的には今の制約になりました。（愚直が通るのは許してください...）

### C

自分たちの想定解が 350ms だったのと pypy の方が TLE していたのを見て、このままだと怒られるのではと思い急遽 tl を 2.5s に変更しました（これによる AC は今現在でも増えていません）。

### D

難易度 2 なので簡単枠だろ！ と思っていましたがびっくりするくらい解かれなくて当時焦っていました。

### G

傑作ができたと思っています。最後の式変形をさぼって二分探索で通すこともできますが、まじめに詰めるととてもシンプルな式になって凄い美しいと思っています。全人類二分探索で解かれていたので悲しい気持ちになりましたが...

### H,I

もともとは、「整数列が与えられるので、連続とは限らない部分列にしとしとと数列がいくつあるか求めてください（N ＜＝ 2000）」を提案していました。そのついでで「しとしとと数列を使った簡単枠作れない？」と聞いたところ Bread が H、僕が I を提案してました。そしてもともとの問題が簡単すぎて爆破されたので H,I を代わりに両方置くことにしました。流石に両方簡単枠だとバレるとあれなので見た目だけでも難しくしようと努力しました。その結果 H の AC がだいぶ伸びなくてえ？ になりました。（最終的な AC 数は I と同じくらいになりました）

### J

N ＜＝ 2000 にするか N ＜＝ $2 \times 10^5$にするか $Bread$ と長い間議論をしていましたが、結果実装がそこまで重くないということで $2 \times 10^5$にしました。

## 振り返り

かなり新入生にとっては重いコンテストだっただろうと思っています。が、いつかこのレベルの問題たちを倒せるようになってほしいなという願望（要求）で作っていたので許してほしいです。また、久しぶりにコンテストを開くことになったので作問こんなに苦

労するんだな～と改めて実感していました。あと戒めとして解説は早めに書こうと思います。

## 終わりに

殴り書きになりましたが書きたいことは全部書けたと思っています。お読みいただきありがとうございました。

# APC の入塾テストと JOI 春合宿の参加記

78th goldenpotato

中三 78 期の翁越司です。APC の入塾テストと JOI 春合宿に参加したので参加記を書きます。

## APC 入塾テスト

APC は AtCoder Programming Class のことで AtCoder の競技プログラミング教室です。

入塾テストは今年 2 月に行われて、算数の問題が大問 10 個出されました。各大問は小問 2 つで配点は 3 点と 7 点です。50 点とれたら必ず合格すると言われましたが、実際の合格点は 30 点でした。

私の得点は 28 点でした。

原因としては大問 1 で数列を数百項目まで書き出して時間がなくなったのが問題だと思います。

他の問題も全く解き方が分からず、解けたと思った問題も計算ミスで得点になりませんでした。

来年こそは合格したいので算数を学び直し、普段からちゃんと塾の宿題をさぼらずにやって計算ミスを減らすべきだと思います。(他人事)

## JOI の春合宿

JOI の春合宿にも参加しました。今はファイナルステージという名前です。

4 日間毎日 5 時間問題を 3 問ずつ解きます。部分点がたくさんあります。

パソコンは用意されていて、キーボードとマウスは持参する人がたくさんいました。

普段からファイルなどの操作に慣れておらず、手軽なオンラインコンパイラに頼っていたので、コンパイルや実行のコマンドを覚えました。

問題は難しかったです。バッチという種類の問題は特に難しくて全く分からなかったの

で解けそうなコミュニケーションの部分点を考えていました。

あまり回答形式に慣れていませんでしたが、やっているうちに少し慣れました。

5 時間の競技が終わるとみんなで昼ご飯を食べました。弁当が配られます。ここで JOIG の参加者と合流します。食べている間にその日の全員の成績表が配られるので競ってもらいに行きます。

食べ終わると解析の時間が始まります。正直言って何の時間なのか分かりませんでした。

その後、作問者による解説が行われます。部分点の解法を理解するのでいっぱいでした。解説されても分からないので問題の難しさが分かります。難しすぎて毎回頭が少し痛くなりました。

全体としてとれる部分点はとったと思いますが、もっととれそうな部分点はあったので、もっと実力をつけたいです。30 人中 26 位でした。

# 文化祭を支える？ 技術

76th kinugoshi8928

## はじめに

初めまして、76th の大谷遥太です！ パ研では高校会計をしています。

今回は、私が所属している文化祭実行委員会の IT 部門で取り組んでいることを紹介してみようと思います。

と聞くと真面目な記事に思えますが、残念ながら筆者が真面目な記事書けないのであんまり真面目に書きません 許してね

## 文化祭実行委員会/IT 部門って?

文化祭実行委員会 (文実) とは、その名の通り文化祭を実行する委員会です。そのうち高校 1,2 年生の文実 25 人はいろいろな部門に分かれて文化祭の準備をしています。文化祭を彩る装飾部門や財布の紐を握る財務部門など華々しい部門がある中で、今回紹介したいのが IT 部門です。

IT 部門は、これまたその名の通り文化祭の IT に関わる部分を他の部門や団体と協力しながら行う部門です。2024 年にできた文実で一番新しい部門で、今年で 3 年目です。活動内容がかなり近いのでパ研とゆかりが深く、初代部門長はパ研出身だったりします。

## 具体的に何をしているの?

IT 部門では、大きく分けて 2 つの仕事があります。**HP** と、**それ以外**です。

まあというのは言い過ぎなんですが、それぐらい HP は他の仕事と毛色が違ったりします。HP は美術部門 (デザイン担当) などと協力して、外部に文化祭の魅力を伝える仕事です。逆にそれ以外は、魅力を伝える/高めるというよりは利便性を上げる仕事です。例としては、器材管理システムやシフト調整ツールなどがあります。

# HP のつくりかた?

ここからはちょっと技術的な話をしようと思います。

ちなみに執筆時点で HP はまだ完成してません たすけて

昨年までは React + react-router-dom(今だと react-router らしい 老人会) を使っていました。これは Single Page Application(SPA) 用のライブラリです。

SPA とは何かというと、一般的なページ遷移をせず、Javascript で内部的に表示するページを切り替える Web サイトです。利点としては、

- ページ遷移が早い
- アニメーションが作りやすい
- ネイティブアプリに近い感覚で作れる

などがあります。逆に欠点として、
ページアクセス → Javascript を取得 → ページの表示に必要なリソースを取得 → 表示
という手順を踏むため、初回アクセスが遅い点が挙げられます。
文化祭の魅力を伝える HP としては初回アクセスが遅いのは少し厳しいので、今年は Astro というフレームワークを使ってみました。
というのは全部言い訳で、私が Astro を使ってみたかったから使いました (え?)

## Astro とは?

Astro とは、React のような書き心地を Multi Page Application + 事前レンダリングで実現できるフレームワークです。事前にレンダリングした結果をサーバーから送信することで初回アクセスを早くできます。また、React などの他の Web フレームワークを使用することもできる、デザイン中心の静的な Web サイトを作るにはとても使いやすいフレームワークです。

Astro を使っていて地味に嬉しかったのが、コンポーネントごとに css のスコープがあることです。React + 素の css だとこの機能がなくて、自分で top-top-first-sub-container のように prefix を付けていかなければならないため、非常に大変でした。

この苦労が減ったのがだいぶ大きかったです。

### ホスティング

Web サイトを作ったら次はホスティングして公開するんですが、文実では伝統的に lolipop というレンタルサーバーを使用しています。この時代にレンタルサーバーってメリットあるの？ って思った方も多いと思うんですが、あんまりないです。多分 Cloudflare Pages とか使った方がアクセスも早いし安いので、今後変わっていくような気がします。

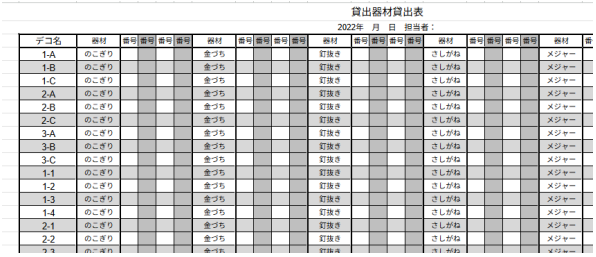
## 器材管理システム

### 器材管理システムとは?

器材管理システムとは、その名の通り器材を管理するシステム..... なのですが、これの成り立ちをきちんと説明しようとすると結構長くなります。

文実では、団体に文化祭作業に必要な器材を貸し出す文実貸出というサービスがあります。

文実貸出で貸し出された器材の管理は、長らく器材に付けられたラベルの番号を紙にメモする形式で行われていました。


貸出器材貸出表

2022年　月　日　担当者：

| デコ名 | 器材 | 番号 | 番号 | 番号 | 番号 | 器材 | 番号 | 番号 | 番号 | 番号 | 器材 | 番号 | 番号 | 番号 | 番号 | 器材 | 番号 | 番号 | 番号 | 番号 | 器材 | 番 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1-A | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-B | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-C | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-A | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-B | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-C | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 3-A | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 3-B | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 3-C | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-1 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-2 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-3 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 1-4 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-1 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-2 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |
| 2-3 | のこぎり | | | | | 金づち | | | | | 釘抜き | | | | | さしがね | | | | | メジャー | |

図 1: 去年まで使われていた貸出メモ用紙。クソちっちゃい

ただ、この制度には、

- メモ用紙がとても小さく、場所を間違えやすい
- 昔の番号と取り違えやすい
- なので、文実側のミスか団体側のミスかわからない

などの問題点がありました。

これで去年ひと悶着起こったりしたんですが、それは本題ではないので飛ばします。
ということで生まれたのが、器材管理システムです。




図 2: 器材に張り付けているバーコード

バーコードを器材に張り付けて、そのバーコードを読み取って貸出/返却処理を行います。人為的なミスの発生頻度が減って、管理もしやすくなりました。

ちなみに代償としてバーコードラベルを張り付ける業務が IT 部門に飛んできました ごめん後輩

こんな器材管理システムなんですが、ここはパ研の部誌なので技術的な話をしていこうと思います。

## 目的

システムをシンプルにして、後輩に引き継ぎやすくすることを目標にしました。
でも Lightsail でめんどくさいことをしちゃったせいで全部台無しになった気がします。
せめて docker は使うべきだったかも
多分現代だとサーバーレスにしちゃうのが一番保守しやすいような気がします。

## バックエンド

バックエンドには hono + node.js + prisma を採用しました。採用理由としては「みんな知ってて保守しやすそうだから」です。面白くないですね〜

DB には SQLite を採用しました。読み込み/書き込み件数が少ないため SQLite でも捌けて、1 ファイルなのでバックアップが取りやすいためです。昔はよく MariaDB を使ってたんですが、使ってみたら SQLite が結構性能良くて別にパフォーマンス重視しないならこれでもいいな〜という気持ちになりました。

### 認証

器材の貸し借りには当然認証が必要なんですが、このシステムでは管理者パスワード 1 本にしています。

そもそも文実の人しか使わない設計なので全員分ユーザーを用意する意味があまりないのと、Google ログインなどは OAuth API を維持するのがちょっと面倒なためです。

### フロントエンド

React + Vite + MaterialUI を採用しました。先ほど少し触れましたが React はこのような Web アプリケーションで使うとメリットが大きいです。現代だと適当に UI を作るときには UI ライブラリを使わないことも多いとは思うんですが、私が MaterialUI の見た目が好きなので採用しました。(え?)

### Web サーバー

Caddy という Web サーバーを採用しました。Let's Encrypt から証明書を取得して自動で https 化してくれる、非常に便利な web サーバーです。設定も Caddyfile のみで済むため、https 化のめんどくさいところを 9 割消してくれます。本当に神です。

### サーバー

AWS Lightsail 上に構築しました。オーソドックスな Linux サーバーなので扱いやすいかな〜と思ったんですが思ったよりめんどくさいことが多くなってしまったのでこれは失敗だったと思います。実行ユーザーを分けるとか、ディレクトリのアクセス権限を制御するとか、昔はよくやってたんですが docker に慣れた状態でやるとめんどくさいです。API サーバーは systemd の service として起動しました。これは結構便利です (過去に minecraft サーバーを systemd の service 化する記事を書いたような気がします)。

# おわりに

原稿の期限が迫ってきたのでこの辺で終わりにしようと思います。

文化祭を表裏両方から支える IT 部門の活動に興味を持ってくれた方は、ぜひ文実になって IT 部門に来てください！

まだまだ発展途上の部門なので、やれることが多くて楽しいと思います！

最後まで読んでくださりありがとうございました！　‘‘‘

# おわりに

76th Magentor

電脳 2026 を最後までご覧いただきありがとうございます。高校パ研部長の小田です。

76 期は、新型コロナウイルスによる規制が緩和されつつあった中一の春に入部しました。それから 5 年が立ち、多くの新入生を迎え、様々な大会にも出場しました。そんな 76 期も、ついに引退が近づいてきました。

この部は、基本的に個人に活動を任せていて、また、部活を卒業後にどのようなことをするのか、についても人ごとに様々です。競技プログラミングにそのまま打ち込む人、ゲームを作る人、研究をする人、あるいは全く別の道に進む人もいます。

とすると、パ研の部員たちの繋がりは、部を卒業して違う場所で違うことをしていたとしても、「パ研の部員であった」という共通点があるからこそ、また集まったり、何かを相談することもできます。そのようなつながりが部を引退した後、そして学校を卒業した後にも残り続けることこそが、このパ研の大きな価値の一つではないでしょうか。

この部誌は、そんなパ研での活動の一つの記録となっています。毎年の部誌には、単に理論的なことだけでなく、部員たちがその時何を考えていたか、どのようなことに取り組んでいたか、といったことまで含めて、その時の状況が色濃く残るものとなっています。今は何気なく書いているこの文章も、何年か後には、おそらくその頃を思い出すきっかけの一つになるのでしょう。

忙しい中それぞれの記事を寄稿してくれた部員たち、また新しい試みやゲームの作成に協力してくれた部員たち、そしてなによりこのデコを作り上げたデコ責 2 人に感謝の意を述べ、この部誌を締めたいと思います。‘‘‘

筑波大学附属駒場中・高等学校
中高パーソナルコンピューター研究部
2026 年度部誌

# 電脳 2026　第 75 回文化祭号

2026 年 10 月 30 日発行

制作・著作　Paken